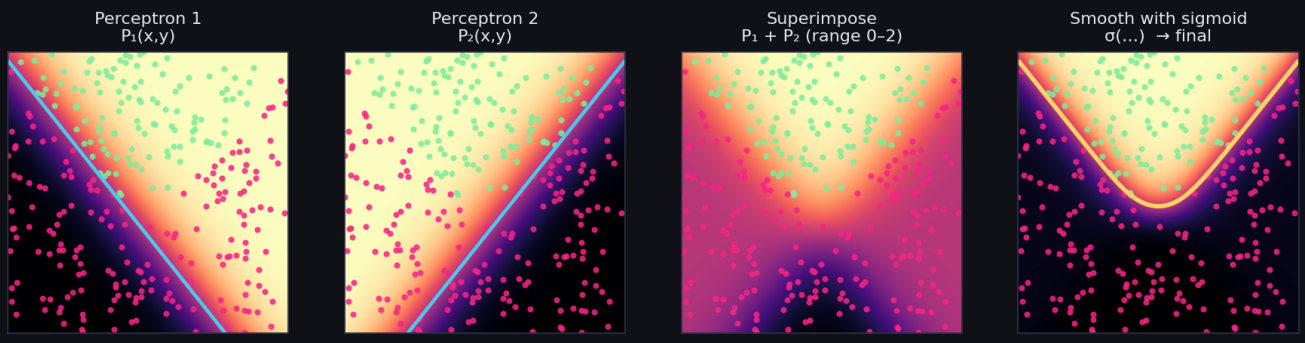
NEURAL NETWORKS • DEEP LEARNING COURSE

Multi-Layer Perceptron & Forward Propagation

From a single neuron to a deep network — a complete, visual, from-scratch tutorial, from basic to pro level.



Two perceptrons (left) are superimposed and smoothed into one curved decision boundary (right) — the core idea of this tutorial.

You will learn	Where
Why a single Perceptron fails on non-linear data	Part 1
The Perceptron with a sigmoid (logistic-regression style)	Part 2
How to build a Multi-Layer Perceptron from first principles	Part 3
The 4 ways to change a network’s architecture	Part 4
Notation: naming every weight, bias and output	Part 5
Forward propagation with matrices (how a network predicts)	Part 6
Pro level: activations, batching, loss, backprop preview, Keras	Part 7
Summary, cheat-sheet, exercises with answers, glossary	Part 8

Contents

How to use this tutorial 4

Part 1 — The problem: why one Perceptron is not enough 4

1.1 Where we are in the course 4

1.2 The problem in one picture 4

1.3 Today's road-map 5

Part 2 — The Perceptron with a sigmoid 5

2.1 An important clarification 5

2.2 The two formulas you must know 6

2.3 Worked example — placement prediction 7

2.4 The geometry: what the line really does 8

Part 3 — Building a Multi-Layer Perceptron from scratch 9

3.1 Step 1: train two different Perceptrons on the same data 9

3.2 Step 2: the abstract idea — superimpose, then smooth 9

3.3 Step 3: the mathematics of "superimpose + smooth" 9

3.4 Step 4: add flexibility with weights and a bias 10

3.5 Step 5: the "aha" — the combiner is itself a Perceptron! 11

3.6 Drawing the complete network 11

3.7 Names of the layers 12

Part 4 — Architecture: four ways to reshape a network 12

4.1 Way 1 — increase nodes in the hidden layer 12

4.2 Way 2 — increase nodes in the input layer 14

4.3 Way 3 — increase nodes in the output layer 15

4.4 Way 4 — increase the number of hidden layers 16

4.5 Why neural networks are "universal function approximators" 16

4.6 Counting trainable parameters 17

Part 5 — Notation: naming every weight, bias and output 17

Part 6 — Forward propagation: how a network makes a prediction 19

6.1 Big picture: forward vs backward 19

6.2 Our example network and its parameter count 19

6.3 The formula for one Perceptron (revision) 20

6.4 Layer 1 in detail 21

6.5 Layer 2 22

6.6 Layer 3 (output) 22

6.7 A complete numerical walk-through 22
6.8 The compact (chained) formula 24
Part 7 — Pro level 25
7.1 Activation functions beyond the sigmoid 25
7.2 Processing many rows at once (batches) 25
7.3 Backpropagation preview (what comes next) 25
7.4 The same network in Keras (3 lines of real work) 26
7.5 Practical pitfalls & professional habits 26
7.6 Hands-on demo: TensorFlow Playground 27
Part 8 — Summary, cheat-sheet, exercises & glossary 27
8.1 The whole story in ten lines 27
8.2 Cheat-sheet 27
8.3 Practice exercises 28
8.4 Glossary 29

How to use this tutorial

This document is a cleaned-up, expanded version of two lecture transcripts (the **Multi-Layer Perceptron intuition** video and the **Forward Propagation** video). The spoken scripts were rearranged into a logical order, every number and example from them is kept, missing diagrams were drawn, and extra explanations, formulas and "pro" sections were added. Nothing from the original talks was removed.

● **Colour code used in every diagram**
● Green = input / correct-class points ● Cyan = hidden-layer perceptrons ● Purple = second hidden layer ● Magenta = output / the other class ● Yellow = weights, decision boundary & key formulas

✓ **Prerequisites (all explained as we go)**
Basic algebra (equation of a line), what a function is, and the idea of a probability between 0 and 1. If you remember what a matrix is, great — we revise matrix multiplication in Part 6.

→ **Reading path**
Beginner: Parts 1–3 | **Intermediate:** Parts 4–6 | **Pro:** Part 7 | **Revision:** Part 8. Look for the boxes titled *Intuition*, *Math*, *Worked example* and *Pro tip*.

Part 1 — The problem: why one Perceptron is not enough

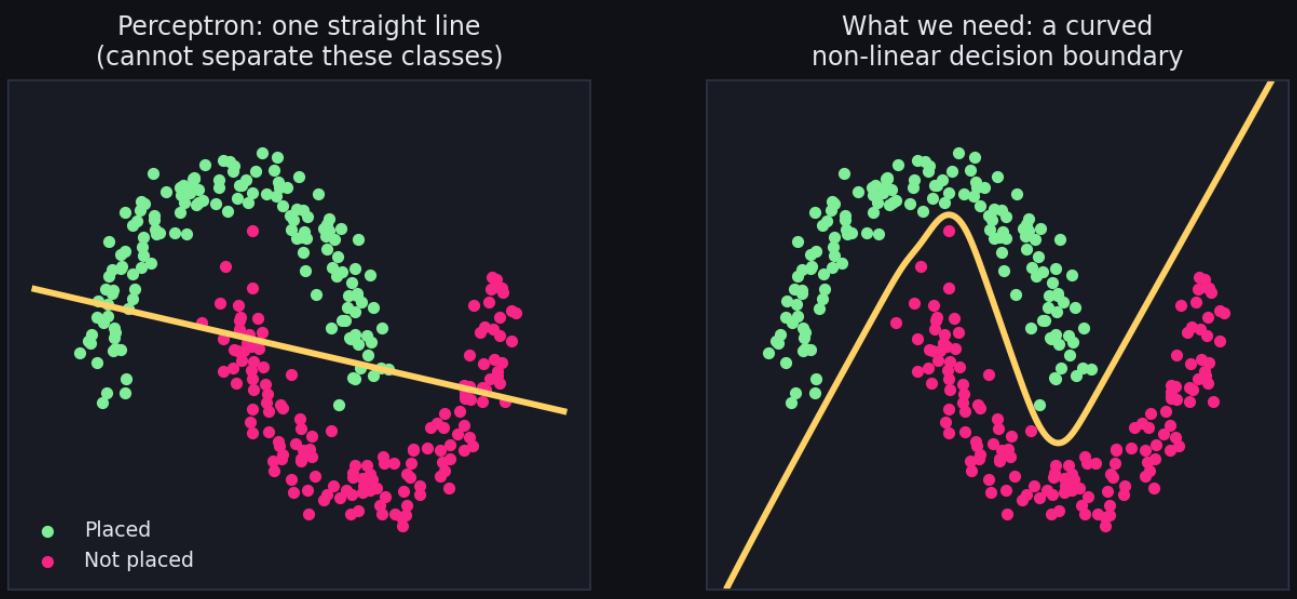
1.1 Where we are in the course

In the previous lesson we learned the **Perceptron** — the smallest building block of a neural network — and found its big weakness: **a Perceptron can only draw a straight line** (a *linear* decision boundary). If your data cannot be split by one straight line, a Perceptron fails.

The cure is the **Multi-Layer Perceptron (MLP)**: join many Perceptrons together into a network. Such a network behaves like a **universal function approximator** — given enough neurons and training it can create *any* kind of non-linear decision boundary. This lesson is, in the true sense, your **first meeting with a real Neural Network**: you will see *why* it can bend its boundary, building the logic completely from scratch (first principles).

1.2 The problem in one picture

Imagine two classes of students: **placed** (green) and **not placed** (magenta). In the picture below the classes are curled around each other. No single straight line can separate them; we need a **curve**.



Left: a Perceptron's straight line cuts through both classes. Right: the curved boundary we want. A non-linear boundary is what we must build — using Perceptrons themselves.

◎ Our challenge
Build an algorithm that captures **any** non-linearity — and the twist: we may only use **Perceptrons** as building blocks. The trick will be to **combine several Perceptrons** so that their straight lines together create a curve.

1.3 Today's road-map

- Part 2: make the Perceptron output a **probability** instead of a hard 0/1 (this is essential for combining them).
- Part 3: take two such Perceptrons, **superimpose** their outputs and **smooth** the result — first as an idea, then mathematically.
- Part 3 (end): realise the combiner is itself a Perceptron ⇒ we have invented the **Multi-Layer Perceptron**.
- Part 4: change the architecture (more nodes, more layers) to capture even harder shapes.
- Part 6: learn forward propagation — how the network actually computes a prediction using matrices.
- Part 7: a hands-on demo idea in **TensorFlow Playground** plus professional-level topics.

Part 2 — The Perceptron with a sigmoid

2.1 An important clarification

Throughout this lesson the Perceptron is **not** the old-style one with a step function. We use:

	Old Perceptron (last lesson)	Perceptron used in this lesson
Activation function	Step function → output 0 or 1 (or −1/+1)	**Sigmoid** → output a probability in (0, 1)
Loss function	Hinge loss	**Log-loss** (binary cross-entropy)
Behaves like...	a hard line classifier	**Logistic Regression**

In other words: **we use our Perceptron like Logistic Regression**. This clarity is very important, because probabilities (numbers between 0 and 1) can be smoothly added and combined, while hard 0/1 answers cannot.

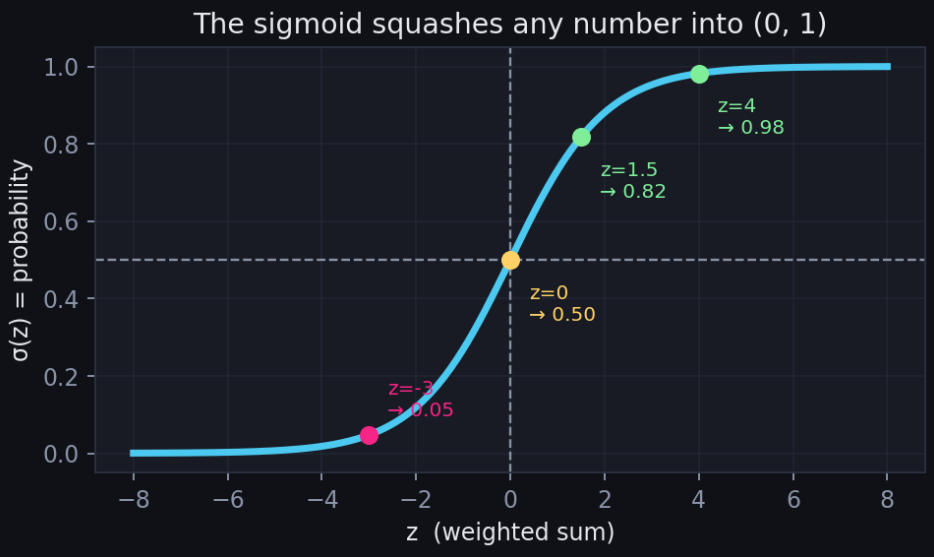
2.2 The two formulas you must know

Step 1 — the weighted sum (called z): multiply every input by its weight, add them, then add the bias *b*.

$$z = w_1x_1 + w_2x_2 + b = \sum_{i=1}^{n} w_ix_i + b$$

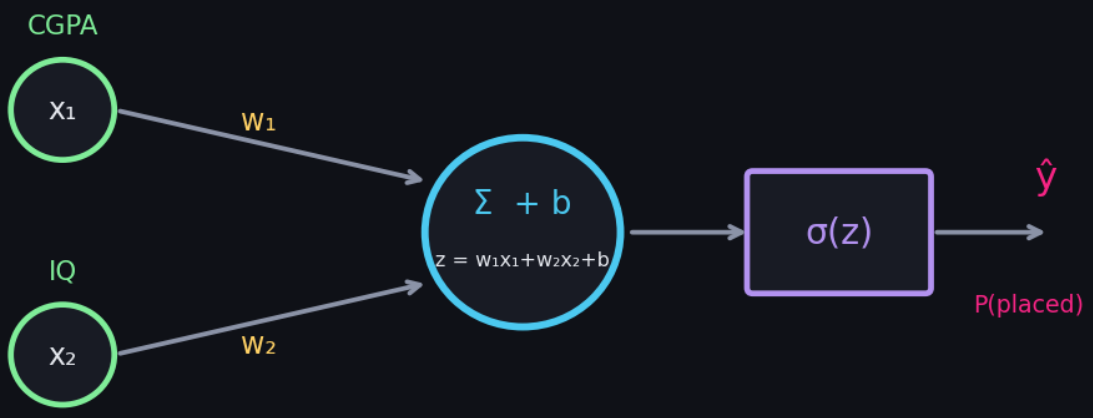
Step 2 — the sigmoid: squash z into a probability.

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$



The sigmoid curve. Large negative z → almost 0. z = 0 → exactly 0.5. Large positive z → almost 1.

◆ Intuition
z can be any number from −∞ to +∞ (it's just a weighted sum). A probability must live between 0 and 1. The sigmoid is a smooth "squashing" machine that converts any z into a valid probability. **σ(0) = 0.5** — right on the fence.



Anatomy of one Perceptron with sigmoid: inputs × weights → add bias (z) → sigmoid → probability ŷ.

2.3 Worked example — placement prediction

The famous example: we have two input columns, **CGPA** and **IQ**, and we want to predict whether **placement happens or not**. Suppose training gave:

$$w_1 = 5, \quad w_2 = 10, \quad b = 3 \;\Rightarrow\; \text{decision line: } 5x + 10y + 3 = 0$$

A new student arrives with **CGPA = 8.7** and **IQ = 87**. First calculate z:

$$z = 5(8.7) + 10(87) + 3 = 43.5 + 870 + 3 = 916.5$$

Then pass z through the sigmoid to get a probability. In the lecture the example result is taken as **0.8** (illustration).

> ⚠ **Reality check (added note)**
> With these raw inputs z = 916.5 is huge, so σ(916.5) ≈ 1.000 — the sigmoid has *saturated*. That is a hint that real inputs should be **scaled** (e.g. CGPA/10, IQ/100) before training, which we use later in the numerical forward-propagation example. For the explanation below, keep the lecture's illustrative value P(placed) = 0.8.

Two probabilities per student. Because only two outcomes exist, the probabilities must add to 1:

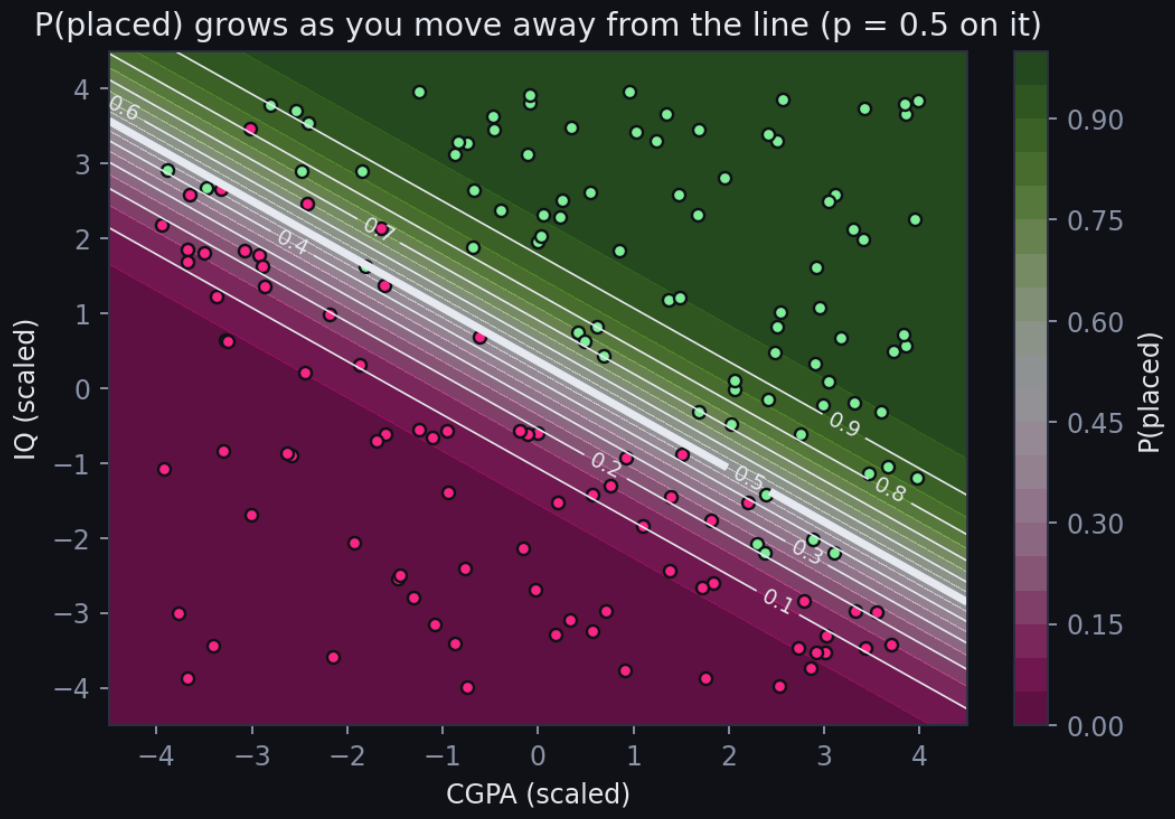
$$P(\text{placed}) = 0.8 \;\Rightarrow\; P(\text{not placed}) = 1 - 0.8 = 0.2$$

So for *every* student you get two numbers — a probability of **Yes** and (by subtracting from 1) a probability of **No**.

2.4 The geometry: what the line really does

Every dot in the data space is a student. The Perceptron's line divides this entire space into two regions — a **green side** (placed more likely) and a **magenta side** (not placed more likely). The probability is not just "green or magenta"; it is a smooth **gradient**:

- **On the line:** P(Yes) = P(No) = 0.5 (the model is completely undecided).
- **Moving into the green side:** P(Yes) rises — e.g. a student close to the line may have 0.6 (and 0.4 for No); one very far away may have 0.9 (and 0.1 for No).
- **Moving into the other side:** P(Yes) drops — e.g. 0.4, then far away very low and P(No) very high.
- Draw lines **parallel** to the decision line: on the green side they carry P(Yes) = 0.6, 0.7, 0.8, 0.9; on the other side 0.4, 0.35, 0.3, 0.2, 0.1 (as in the lecture's sketch).



Probability map of a sigmoid Perceptron. The thick white line is p = 0.5; parallel contour lines show how confidence grows with distance from the line.

★ Key takeaway of Part 2
A sigmoid Perceptron gives, for every student, a pair of probabilities (Yes, No). The decision line is where they tie at 0.5, and confidence grows smoothly as you move away from it.

∑ Math — the loss (how the Perceptron is judged)
With sigmoid we train using **log-loss** for one example (true label y ∈ {0,1}, prediction ŷ):

$$L(y, \hat{y}) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}) \right]$$

If y = 1 the loss is −log(ŷ): tiny when ŷ is near 1, huge when ŷ is near 0. If y = 0 the loss is −log(1−ŷ). Wrong-and-confident predictions are punished heavily.

Part 3 — Building a Multi-Layer Perceptron from scratch

3.1 Step 1: train two different Perceptrons on the same data

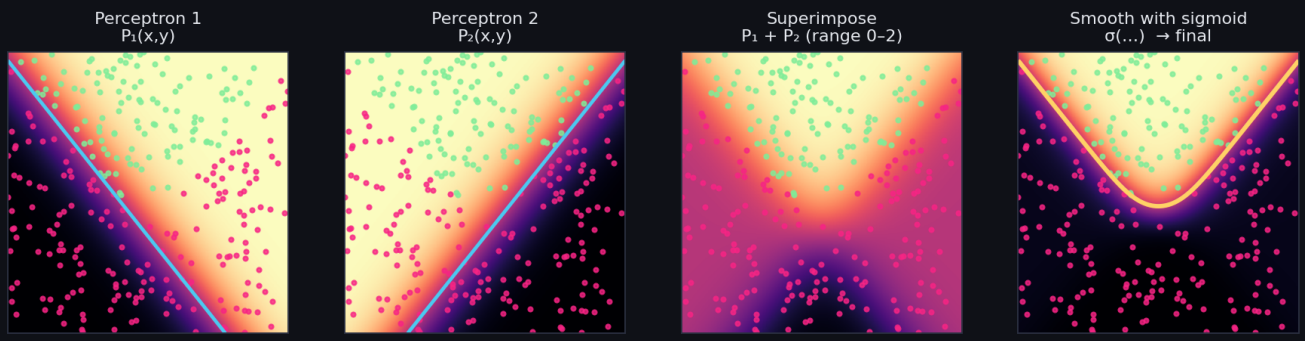
Take the same curved dataset and train **two** different sigmoid Perceptrons on it. Suppose they produced:

	w_1	w_2	bias b	Decision-boundary equation
Perceptron 1	2	3	6	2x + 3y + 6 = 0
Perceptron 2	5	4 (the lecture also mentions 5)	3	5x + 4y + 3 = 0

Each Perceptron draws its **own straight line**. Alone, neither is good enough. What if we join their decisions in some way?

3.2 Step 2: the abstract idea — superimpose, then smooth

Pick up the picture of Perceptron 1 and **superimpose** (place on top of) the picture of Perceptron 2. You get a new decision boundary made of pieces of both lines — a corner or "V". Then apply **smoothing** and the sharp corner becomes a neat curve — the very boundary we wanted.



Superimposition = adding the two probability maps. Smoothing = passing the sum through a sigmoid. Together they bend the boundary.

So far this is only a picture-level idea — **no mathematical logic yet**: we have not said *how* superimposition or smoothing happens. Let's now prove it with real numbers.

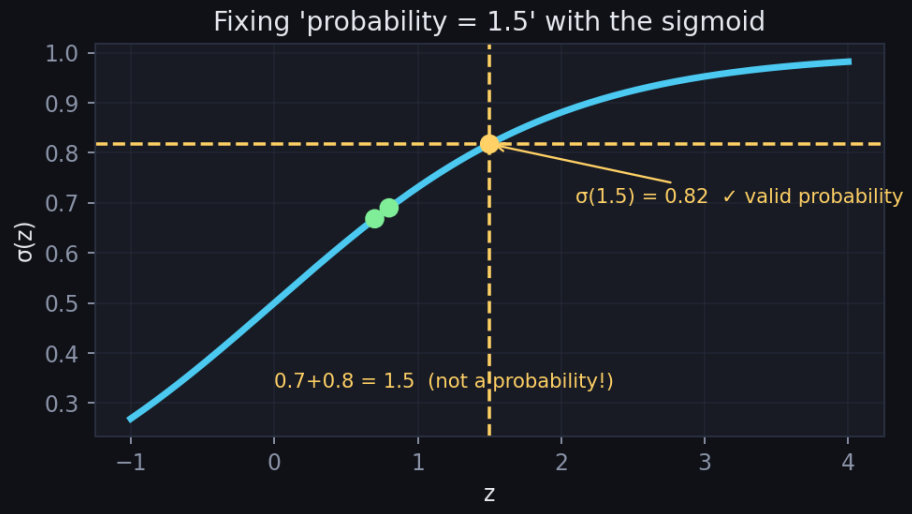
3.3 Step 3: the mathematics of "superimpose + smooth"

Pick one student. Perceptron 1 says P(placed) = **0.7**; Perceptron 2 says P(placed) = **0.8** for the same student.

- **Superimpose = add the two outputs:** 0.7 + 0.8 = **1.5**.
- **Problem:** 1.5 is not a probability (must be between 0 and 1) — a mess!
- **Solution — smoothing = pass it through the sigmoid:**

$$\sigma(1.5) = \frac{1}{1+e^{-1.5}} = 0.82$$

So according to the **combined model**, this student has P(placed) = **0.82**. Repeat for every student: find the probability from Perceptron 1, from Perceptron 2, add, apply sigmoid.



The sigmoid turns the invalid "probability" 1.5 into the valid probability 0.82.

> **◆ What did we really do?**
> **Addition** gave us the superimposition and the **sigmoid** gave us the smoothing. In mathematical language we took a **linear combination** of two Perceptrons and squashed it with a sigmoid.

3.4 Step 4: add flexibility with weights and a bias

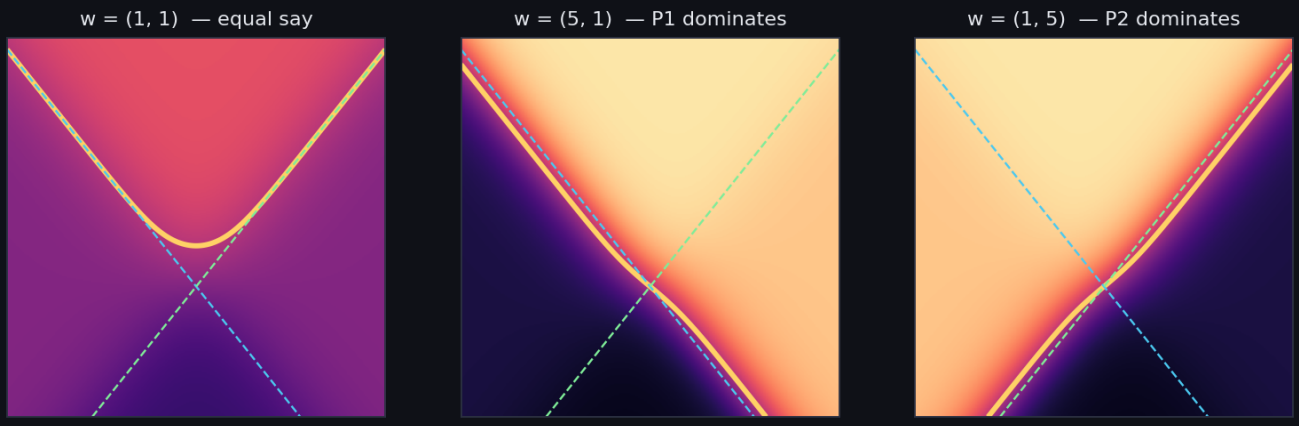
What if we want a boundary where Perceptron 1 has **more influence** than Perceptron 2? Make the addition a **weighted addition**: send each probability through its own weight. Say weight 10 for Perceptron 1 and weight 5 for Perceptron 2 (so Perceptron 1 counts *double*). We may also add a bias of, say, 3.

$$z = 10 \cdot P_1 + 5 \cdot P_2 + 3 \qquad \hat{y} = \sigma(z)$$

For our student with $P_1 = 0.7$ and $P_2 = 0.8$:

$$z = 0.7 \times 10 + 0.8 \times 5 + 3 = 7 + 4 + 3 = 14 \Rightarrow \hat{y} = \sigma(14) \approx 0.999999$$

The sigmoid again returns a probability between 0 and 1 (here very close to 1, because we chose big illustrative weights). We repeat this for every student in the data.



How the combiner's weights reshape the final boundary (yellow). Left: equal say. Middle: Perceptron 1 dominates. Right: Perceptron 2 dominates. Dashed lines are the two original perceptron boundaries.

3.5 Step 5: the "aha" — the combiner is itself a Perceptron!

Look carefully at what we just computed: two inputs (the outputs of Perceptron 1 and 2), two weights (10 and 5), a bias (3) and a sigmoid. **That is exactly a Perceptron** — except its inputs come from *other Perceptrons* instead of from the raw data.

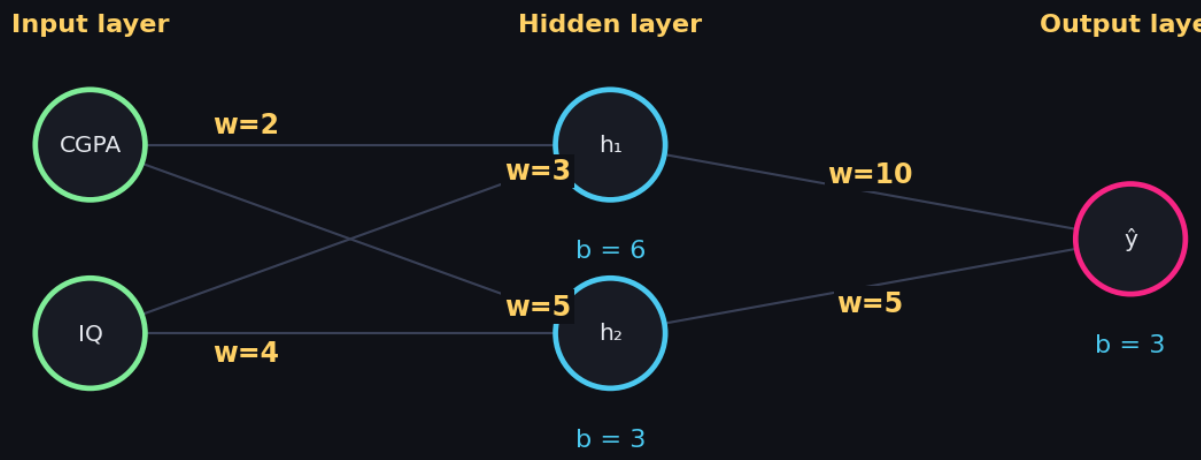
$$\text{Perceptron 1},\ \text{Perceptron 2} \longrightarrow \text{Perceptron 3} \longrightarrow \hat{y}$$

We therefore have a combination of **three Perceptrons** — and that is a **Multi-Layer Perceptron**.

3.6 Drawing the complete network

Let's draw it with nodes, weights and biases using the lecture's numbers:

Node	Receives	Weights	Bias
Perceptron 1 (hidden h_1)	CGPA, IQ	CGPA→h_1 = 2, IQ→h_1 = 3	6
Perceptron 2 (hidden h_2)	CGPA, IQ	CGPA→h_2 = 5, IQ→h_2 = 4	3
Perceptron 3 (output $\hat{y}$)	outputs of h_1 and h_2	h_1→$\hat{y}$ = 10, h_2→$\hat{y}$ = 5	3



The finished Multi-Layer Perceptron. Both CGPA and IQ are fed to both hidden perceptrons; their outputs are combined by the output perceptron.

3.7 Names of the layers

Layer	What it is	In our network
Input layer	Just holds the raw data columns (no computation)	CGPA, IQ (2 nodes)
Hidden layer	Perceptrons between input and output; each draws its own line	Perceptron 1 & 2 (2 nodes)
Output layer	Gives the final prediction $\hat{y}$	Perceptron 3 (1 node)

★ Key takeaway of Part 3
By organising Perceptrons in layers and taking a **linear combination of their outputs followed by a sigmoid**, a network can bend straight lines into curves — that is how a Multi-Layer Perceptron captures non-linearity.

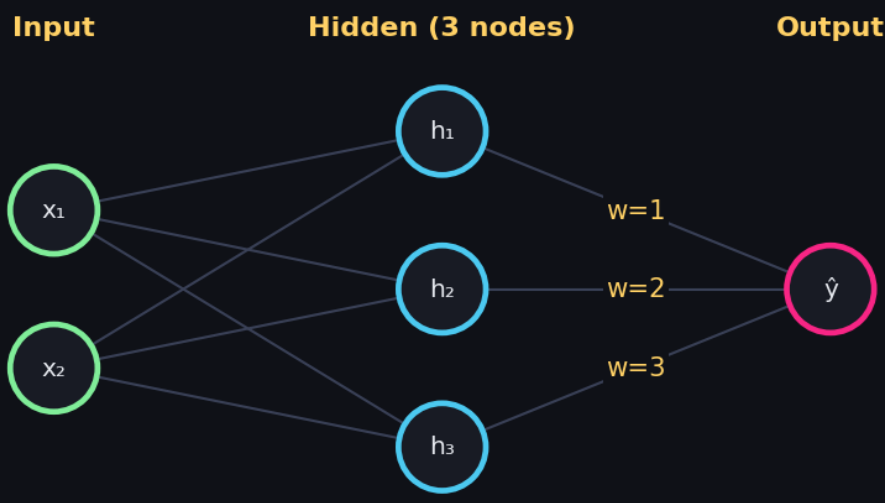
Part 4 — Architecture: four ways to reshape a network

The **architecture** of a neural network means **how its nodes (Perceptrons) are connected** — which nodes exist, in how many layers, and which weights link them. You can change the architecture to suit your data. There are **four** ways.

#	Change	Why / when	Effect
1	More nodes in a hidden layer	Data is highly non-linear	More lines to combine ⇒ more complex boundaries
2	More nodes in the input layer	Your data has more input columns	Higher-dimensional space (planes / hyperplanes)
3	More nodes in the output layer	Multi-class classification	One output per class
4	More hidden layers	Very complex non-linear patterns	Boundaries of boundaries ⇒ deep networks

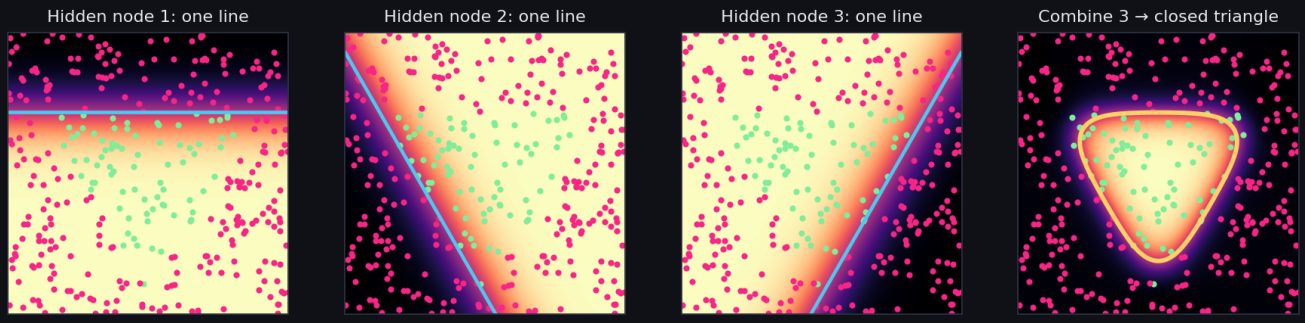
4.1 Way 1 — increase nodes in the hidden layer

The easiest change: add more Perceptrons to the hidden layer. Everything else (the data) is unchanged. In our earlier network the hidden layer had 2 nodes; here it has **3**.



A network with three hidden nodes: one extra Perceptron, one extra weight into the output.

Each hidden node draws its own line. Combining three lines can enclose a **region** (like a triangle) — impossible with a single line, and tighter than what two lines can do.



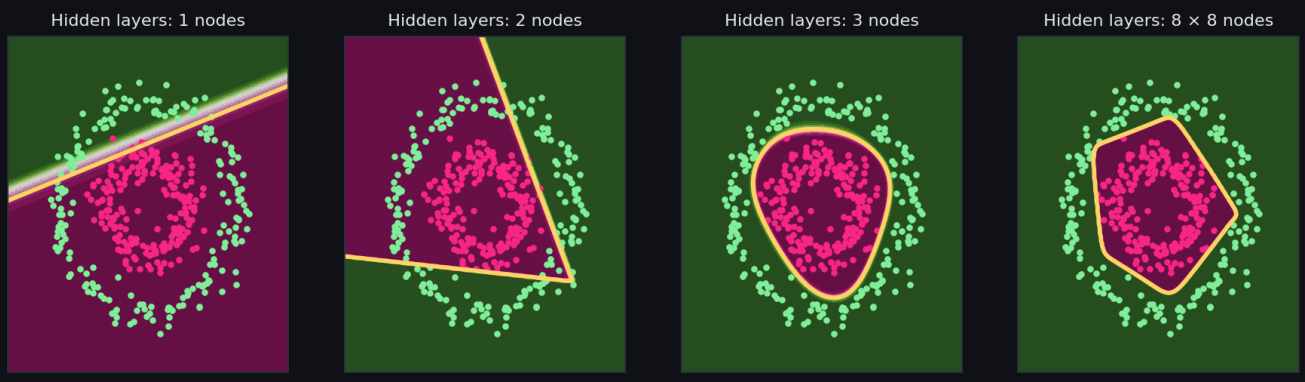
Three straight lines (cyan) combined by the output node form a closed, rounded region (yellow) that wraps the green class.

Math — only one extra term appears. Take one point. The three hidden Perceptrons say 0.2, 0.3, 0.4. The output weights are 1, 2, 3 and the output bias is 5:

$$z = 0.2 \cdot 1 + 0.3 \cdot 2 + 0.4 \cdot 3 + 5 = 0.2 + 0.6 + 1.2 + 5 = 7.0 \;\Rightarrow\; \hat{y} = \sigma(7.0) \approx 0.999$$

★ Rule

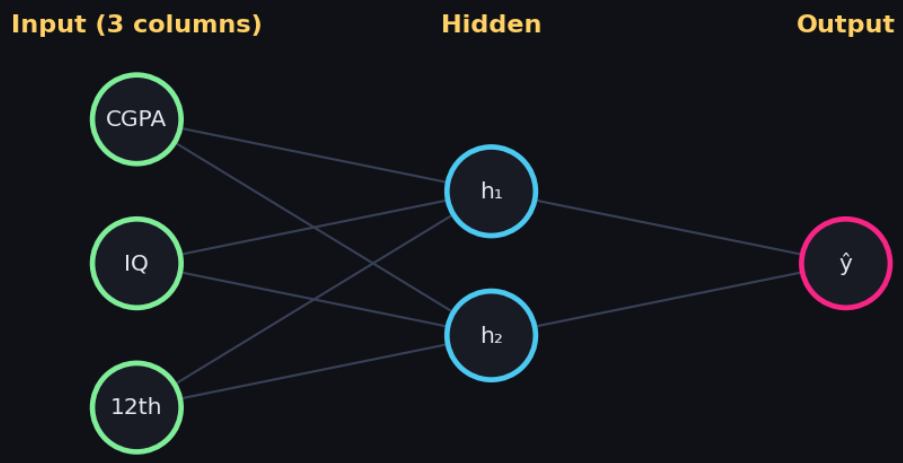
Add one more hidden node ⇒ add one more term (weight × output) when calculating z. Add as many hidden nodes as you like — **more nodes ⇒ more ability to build complex non-linear boundaries**.



Same dataset (a ring inside a ring). 1 hidden node can only draw a line; 2 nodes a wedge; 3 nodes a rounded shape; two layers of 8 nodes a smooth, accurate boundary.

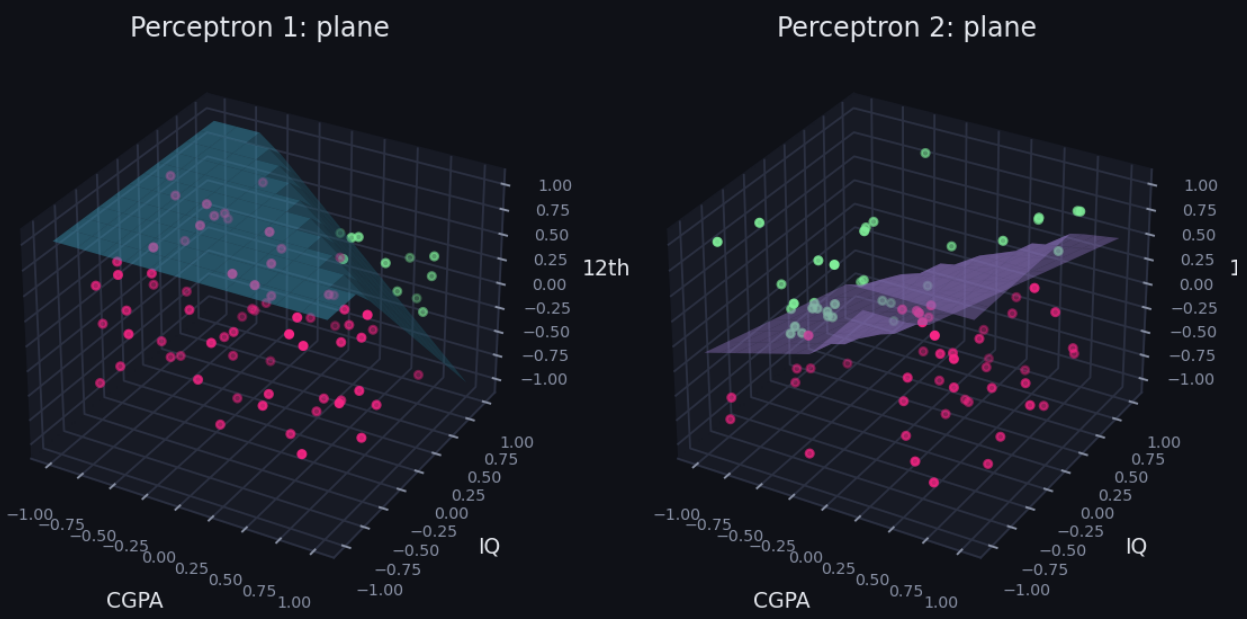
4.2 Way 2 — increase nodes in the input layer

You do this only when your **data gets more input columns**. Example: besides CGPA and IQ, add **12th-grade marks**. Now the input layer has **3 nodes** and the data lives in **3-D space** (x = CGPA, y = IQ, z = 12th marks).



Three input columns ⇒ three input nodes.

In 2-D a Perceptron is a **line**; in 3-D it is a **plane**. Two Perceptrons give two planes; we again take a linear combination (probability × w_1 + probability × w_2 + bias → sigmoid) to get a probability between 0 and 1 for each point.

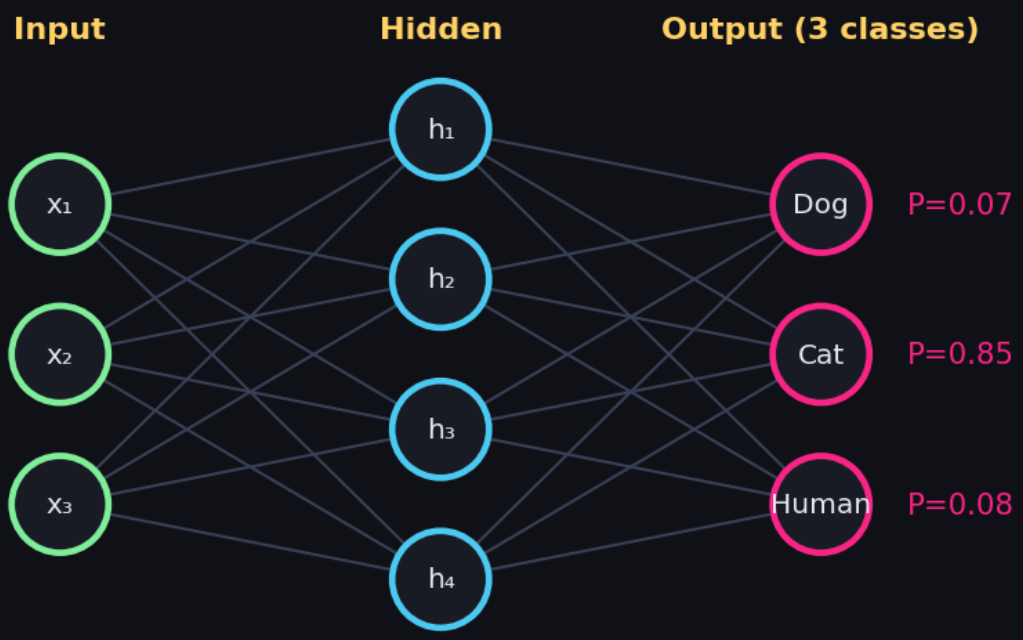


In 3-D every Perceptron's decision boundary is a plane that splits green from magenta points.

> **★ Rule**
> **Number of input nodes = number of input columns.** With 4 columns the space is 4-D and the boundary is a *hyperplane* — we cannot draw it, but the maths is identical.

4.3 Way 3 — increase nodes in the output layer

So far there was always a single output Perceptron. That is perfect for **binary** classification (placed / not placed). For **multi-class classification** you keep **one output node per class**. Example: a photo that may contain a **dog**, a **cat** or a **human** ⇒ three output nodes. The class with the highest probability wins.



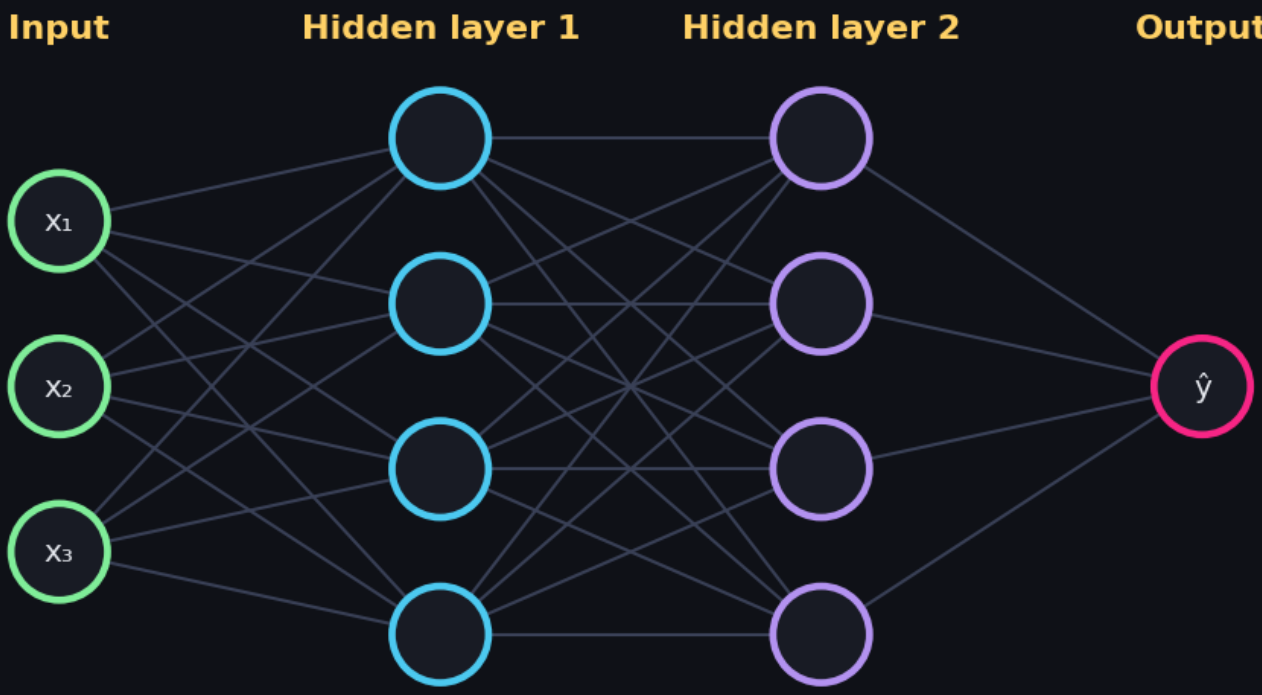
Three output nodes for three classes. The probabilities shown are illustrative; the highest (Cat, 0.85) is the prediction.

∑ Pro tip — softmax
For multi-class problems the output layer normally uses **softmax** instead of independent sigmoids, so the class probabilities add up to exactly 1:

$$\text{softmax}(z_k) = \frac{e^{z_k}}{\sum_j e^{z_j}} \qquad \text{e.g. } z = [1.0,\ 3.0,\ 1.1] \Rightarrow [0.105,\ 0.778,\ 0.116]$$

4.4 Way 4 — increase the number of hidden layers

Until now we only widened one layer. You can also **stack hidden layers**: input → hidden layer 1 → hidden layer 2 → … → output.

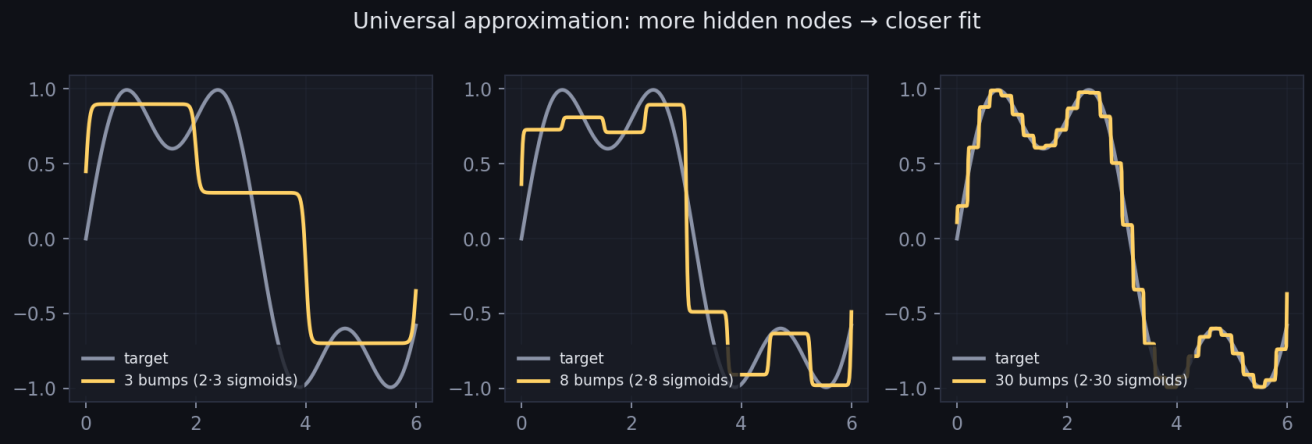


A network with two hidden layers (a "deep" network).

Why is depth useful? The first hidden layer only creates **linear** boundaries. The next layer takes *those* as inputs and combines them into more complex shapes; the layer after that combines the complex shapes again. As you go deeper, the network captures **more and more complex relationships** — each layer builds on the previous one, like words → sentences → paragraphs.

4.5 Why neural networks are "universal function approximators"

Give a neural network **enough hidden nodes/layers** and **enough training time**, and it can learn *any* mathematical relationship, however non-linear. This is the **Universal Approximation** property. A simple 1-D way to see it: two sigmoids with opposite signs make a **bump**; many bumps of different heights can trace any curve.



A target curve (grey) approximated by sums of sigmoid bumps (yellow). More hidden nodes ⇒ a closer fit.

> ⚠ **Trade-offs (real world)**
> Deeper/wider networks need more training time, more data and more care (they can overfit). But in theory: **given enough hidden layers and training time, a neural network can build any function.**

4.6 Counting trainable parameters

Every connection (weight) and every non-input node (bias) is a number the network must learn. For a layer with n_{in} inputs and n_{out} nodes:

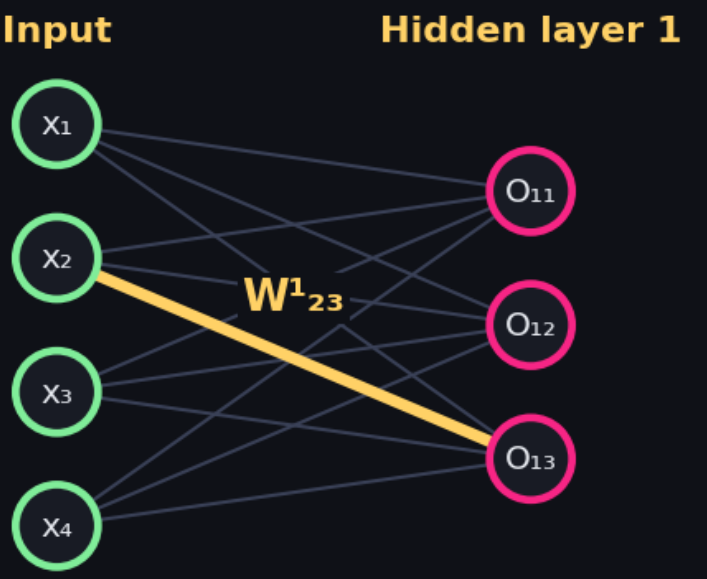
$$\text{parameters of one layer} = n_{in} \times n_{out} + n_{out} \quad \Rightarrow \quad \text{total} = \sum_{\text{layers}} (n_{in}\, n_{out} + n_{out})$$

Network	Calculation	Total
2 → 2 → 1 (Part 3)	(2·2+2) + (2·1+1) = 6 + 3	9
2 → 3 → 1 (Way 1)	(2·3+3) + (3·1+1) = 9 + 4	13
3 → 2 → 1 (Way 2)	(3·2+2) + (2·1+1) = 8 + 3	11
4 → 3 → 2 → 1 (Part 6)	(12+3) + (6+2) + (2+1) = 15 + 8 + 3	**26**

Part 5 — Notation: naming every weight, bias and output

Before forward propagation we need a naming system so that nobody gets confused when the network grows. The convention used in the lectures:

Symbol	Meaning	Example
x_i (or $a^{[0]}_i$)	i-th input feature (activation of layer 0)	x_1 = CGPA
W^l_{ij}	weight in layer *l* going from node *i* of the previous layer to node *j* of layer *l*	W^1_{23}: input 2 → hidden node 3
b^l_j	bias of node *j* in layer *l*	b^1_3 = bias of 3rd node of layer 1
O_{lj} (or $a^{[l]}_j$)	output (activation) of node *j* in layer *l*	O_{21} = 1st node of layer 2
$z^{[l]}$	weighted sum before the sigmoid in layer *l*	$z^1 = W^T a^0 + b^1$
$\hat{y}$ (= $a^{[L]}$)	final prediction of the network	a^3 for a 3-layer net



Weight W^1_{ij} : from input node i → to node j of layer 1

Bias b^1_j : one per node of layer 1

Reading W^1_{23}: layer 1, from input node 2, to hidden node 3 — the highlighted connection.

◆ Memory trick

Rows = where the weight comes from; columns = where it goes to. In the weight matrix W^l, the first index is the source node and the second index is the destination node.

Part 6 — Forward propagation: how a network makes a prediction

6.1 Big picture: forward vs backward

Training a neural network involves two phases that repeat many times:



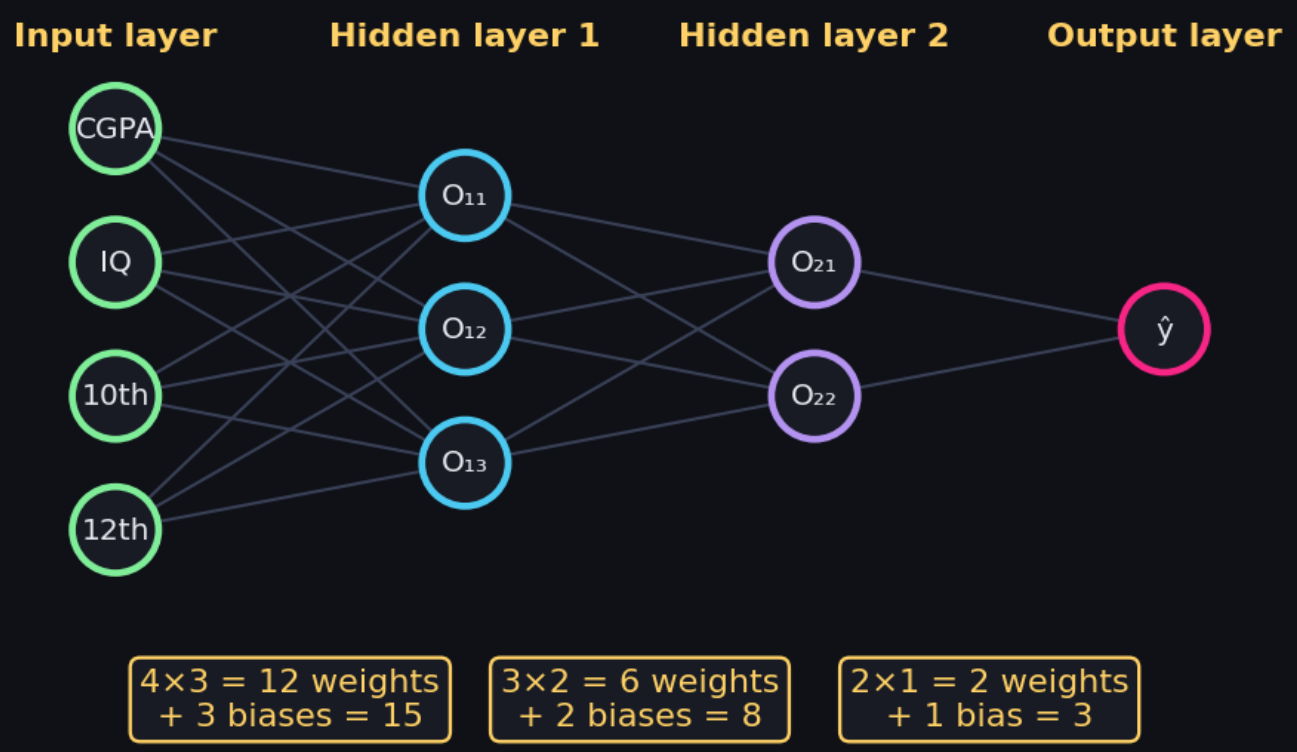
The training loop. Forward propagation (this part) makes the prediction; backpropagation (a later lesson) updates the weights.

- **Forward propagation** — the input moves *forward* through the network, layer by layer, until a prediction ŷ comes out.
- **Backpropagation** — the error is sent *backward* and the weights are updated so training happens.

We study forward propagation **first**, because understanding it makes backpropagation far easier. The beautiful part: **linear algebra handles everything**. No matter how big the architecture is, you only do matrix dot products, add biases, and apply the sigmoid.

6.2 Our example network and its parameter count

Data: 4 input columns — **CGPA, IQ, 10th marks, 12th marks** — and the output column **placed** (yes/no). Architecture: **4 → 3 → 2 → 1** (two hidden layers and one output node). Always count the trainable parameters first — this is how many numbers the backpropagation algorithm will search for.



Architecture 4-3-2-1 with its parameter count: 26 weights and biases.

Layer	Weights	Biases	Parameters
Input → Hidden 1	4 × 3 = 12	3	15
Hidden 1 → Hidden 2	3 × 2 = 6	2	8
Hidden 2 → Output	2 × 1 = 2	1	3
Total	20	6	**26**

At the start of training all 26 values are **random**. We feed rows one at a time. The first row in the lecture is **(7.2, 72, 60, 81)**: 7.2 enters the first input node, 72 the second, 60 the third and 81 the fourth, and ŷ comes out of the single output node.

6.3 The formula for one Perceptron (revision)

$$\text{output} = \sigma\left(\sum_i w_i x_i + b\right)$$

To compute the prediction we do this **layer by layer**. The trick: instead of computing node after node, compute a whole layer at once using a matrix.

6.4 Layer 1 in detail

Weight matrix W^1 — the 12 weights from the 4 inputs to the 3 hidden nodes. Its shape is **4 × 3** (rows = inputs, columns = hidden nodes):

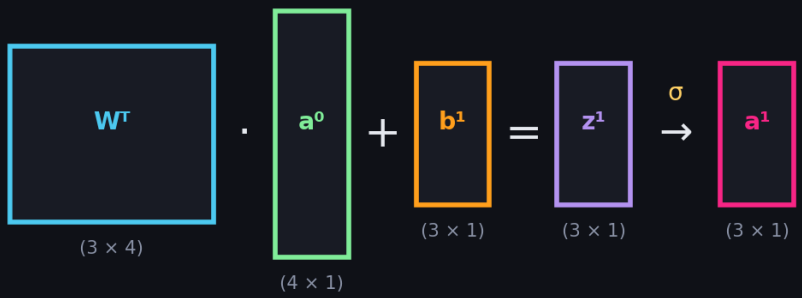
$$W^1 \text{ (weights)} = \begin{bmatrix} W^1_{11} & W^1_{12} & W^1_{13} \\ W^1_{21} & W^1_{22} & W^1_{23} \\ W^1_{31} & W^1_{32} & W^1_{33} \\ W^1_{41} & W^1_{42} & W^1_{43} \end{bmatrix} \quad 4\times3$$

Input (column) vector a^0 — shape 4 × 1. **Bias vector b^1** — shape 3 × 1:

$$a^0 = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \quad 4\times1 \qquad \text{and} \qquad b^1 = \begin{bmatrix} b^1_1 \\ b^1_2 \\ b^1_3 \end{bmatrix} \quad 3\times1$$

We want a 3 × 1 result, but W^1 is 4 × 3 and a^0 is 4 × 1 — the inner sizes (3 and 4) do not match. So we **transpose** W^1 (flip rows and columns) to get W^T of shape **3 × 4**. Now (3 × 4)·(4 × 1) works and gives **3 × 1**. Then add b^1 (3 × 1) and apply the sigmoid:

$$z^{[1]} = (W^{[1]})^T a^{[0]} + b^{[1]}, \qquad a^{[1]} = \sigma(z^{[1]})$$



Shape rule

(3×4)·(4×1) → (3×1)
inner numbers (4,4) must match;
outer numbers give the result shape.

Shape bookkeeping for layer 1: (3×4)·(4×1) + (3×1) = (3×1). Inner numbers must match; outer numbers give the result shape.

Written out, the first row of the dot product is: $W^1_{11}\cdot x_1 + W^1_{21}\cdot x_2 + W^1_{31}\cdot x_3 + W^1_{41}\cdot x_4$ — the weighted sum going into the *first* hidden node. The 2nd and 3rd rows give the other two hidden nodes. The final matrix holds **O_{11}, O_{12}, O_{13}** — the outputs of the three hidden nodes, which become the input of the next layer.

6.5 Layer 2

Hidden layer 2 has 2 nodes and receives 3 inputs (O_{11}, O_{12}, O_{13}), so there are **6 weights**. W^2 has shape **3 × 2**; its transpose is 2 × 3. Input a^1 is 3 × 1; the result is **2 × 1**; the bias b^2 is 2 × 1:

$$a^{[2]} = \sigma\left((W^{[2]})^T a^{[1]} + b^{[2]}\right) \;:\; (2\times 3)\cdot(3\times 1) + (2\times 1) = (2\times 1)$$

6.6 Layer 3 (output)

Output layer: 2 inputs → 1 node, so W^3 has 2 weights (shape 2 × 1), transposed 1 × 2. With a^2 (2 × 1) the product is **1 × 1**; add the 1 × 1 bias b^3 and apply sigmoid. That single number is **$\hat{y}$** (also called a^3):

$$\hat{y} = a^{[3]} = \sigma\left((W^{[3]})^T a^{[2]} + b^{[3]}\right) \;:\; (1\times 2)\cdot(2\times 1) + (1\times 1) = (1\times 1)$$

Look at what happened: your entire row of data rolled in and was converted into **one single number** — purely with matrix multiplication.

6.7 A complete numerical walk-through

Let’s run real numbers. Inputs are scaled to 0–1 so the sigmoid does not saturate (CGPA 7.2/10, IQ 72/100, 10th 60/100, 12th 81/100):

$a^0 = x$

$$\begin{bmatrix} 0.72 \\ 0.72 \\ 0.60 \\ 0.81 \end{bmatrix}$$

4×1

Layer 1

$$\underset{(W^1)^T}{\begin{bmatrix} 0.10 & 0.40 & 0.70 & -0.20 \\ 0.20 & -0.50 & 0.80 & 0.30 \\ 0.30 & 0.60 & -0.90 & 0.50 \end{bmatrix}}_{3\times 4} \cdot \underset{a^0}{\begin{bmatrix} 0.72 \\ 0.72 \\ 0.60 \\ 0.81 \end{bmatrix}}_{4\times 1} = \underset{W\cdot a}{\begin{bmatrix} 0.618 \\ 0.507 \\ 0.513 \end{bmatrix}}_{3\times 1} + \underset{b^1}{\begin{bmatrix} 0.10 \\ -0.10 \\ 0.20 \end{bmatrix}}_{3\times 1} = \underset{z^1}{\begin{bmatrix} 0.718 \\ 0.407 \\ 0.713 \end{bmatrix}}_{3\times 1}$$

check row 1: $0.1(0.72) + 0.4(0.72) + 0.7(0.60) + (-0.2)(0.81) = 0.618\,;\;\; + 0.1 \Rightarrow 0.718$

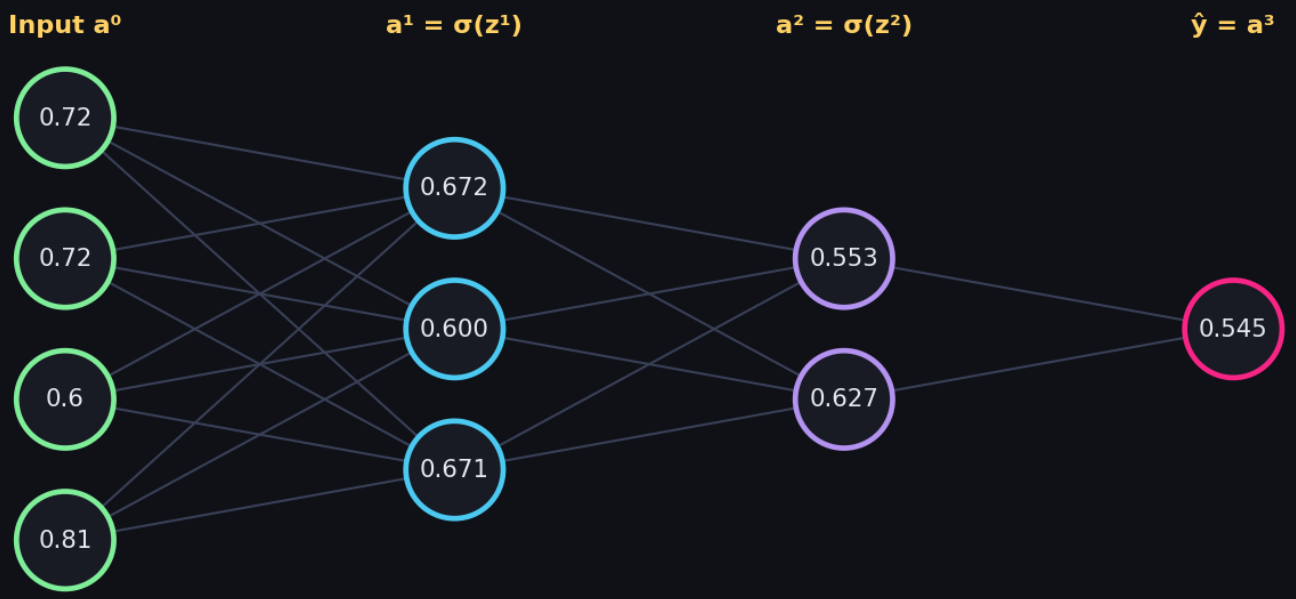
$$z^1 = \begin{bmatrix} 0.718 \\ 0.407 \\ 0.713 \end{bmatrix}_{3\times1} \xrightarrow{\sigma} a^1 = \begin{bmatrix} 0.6722 \\ 0.6004 \\ 0.6711 \end{bmatrix}_{3\times1}$$

Layer 2

$$\underbrace{\begin{bmatrix} 0.50 & 0.30 & -0.60 \\ -0.40 & 0.20 & 0.70 \end{bmatrix}}_{(W^2)^T\ 2\times3} \cdot \underbrace{\begin{bmatrix} 0.6722 \\ 0.6004 \\ 0.6711 \end{bmatrix}}_{a^1\ 3\times1} + \underbrace{\begin{bmatrix} 0.10 \\ 0.20 \end{bmatrix}}_{b^2\ 2\times1} = \underbrace{\begin{bmatrix} 0.214 \\ 0.521 \end{bmatrix}}_{z^2\ 2\times1} \xrightarrow{\sigma} \underbrace{\begin{bmatrix} 0.5532 \\ 0.6274 \end{bmatrix}}_{a^2\ 2\times1}$$

Layer 3 (output)

$$\underbrace{\begin{bmatrix} 0.80 & -0.50 \end{bmatrix}}_{(W^3)^T\ 1\times2} \cdot \underbrace{\begin{bmatrix} 0.5532 \\ 0.6274 \end{bmatrix}}_{a^2\ 2\times1} + \underbrace{\begin{bmatrix} 0.05 \end{bmatrix}}_{b^3\ 1\times1} = \underbrace{\begin{bmatrix} 0.1789 \end{bmatrix}}_{z^3\ 1\times1} \xrightarrow{\sigma} \underbrace{\begin{bmatrix} 0.5446 \end{bmatrix}}_{\hat{y}\ 1\times1}$$



The same forward pass shown on the network: each node displays the number it outputs. The final prediction is ŷ ≈ 0.545 (54.5% chance of placement) — this is a random, untrained network, so it is not yet meaningful.

> ⚠ **Why ≈ 0.54 and not a confident answer?**
> At the beginning of training weights are random, so predictions are essentially random. Backpropagation will adjust all 26 numbers until ŷ is close to the true label for most rows.

6.8 The compact (chained) formula

Call the input the **activation of layer 0**, $a^{[0]}$. The output of layer 1 is $a^{[1]}$, then $a^{[2]}$, and finally $a^{[3]} = \hat{y}$. The same rule is applied again and again:

$$a^{[l]} = \sigma\left((W^{[l]})^T a^{[l-1]} + b^{[l]}\right) \quad l = 1, 2, 3$$

Row-vector convention. In the lecture it is written as $a^0 \cdot W^1 + b^1$. That is the same thing with inputs stored as a *row* (1 × 4) times W (4 × 3) → 1 × 3, which avoids the transpose. Both are correct; just keep the shapes consistent.

You can also nest everything into one expression:

$$\hat{y} = \sigma(\sigma(\sigma(a^{[0]}W^{[1]} + b^{[1]})W^{[2]} + b^{[2]})W^{[3]} + b^{[3]})$$

This is the “complex expression” the network is solving. Add more layers and it just gets longer — but it is a very organised way of doing mathematics, so even a huge architecture stays simple.

> ✎ **Do it yourself (the best practice)**
> Take a notebook and pen. Draw a small network (e.g. 3 → 2 → 1), invent weights, and compute the prediction with matrices. This practice teaches more than re-watching the lecture. Exercises are in Part 8.

Python (NumPy) — the exact forward pass above

```
import numpy as np
sigmoid = lambda z: 1 / (1 + np.exp(-z))

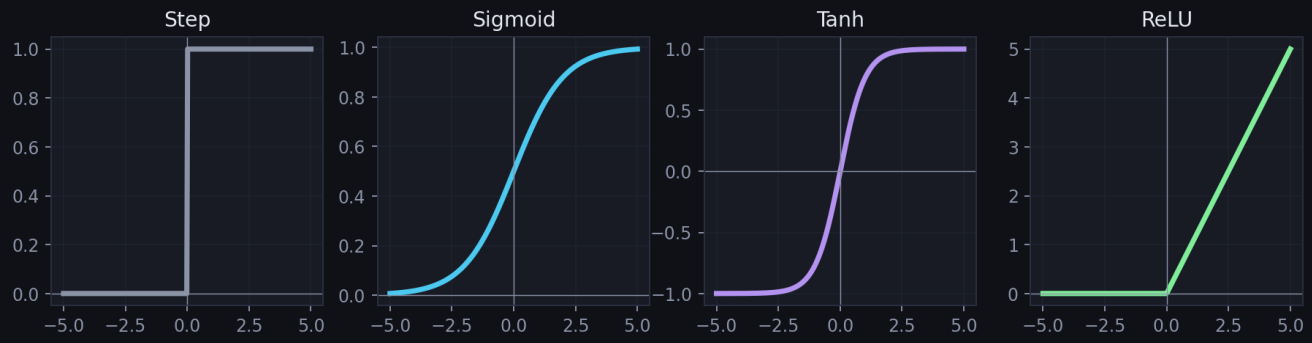
x  = np.array([0.72, 0.72, 0.60, 0.81])            # a0   (4,)
W1 = np.array([[ 0.1, 0.2, 0.3],
               [ 0.4,-0.5, 0.6],
               [ 0.7, 0.8,-0.9],
               [-0.2, 0.3, 0.5]])                   # (4,3)
b1 = np.array([0.1, -0.1, 0.2])                     # (3,)
W2 = np.array([[0.5,-0.4],[0.3,0.2],[-0.6,0.7]])    # (3,2)
b2 = np.array([0.1, 0.2])                           # (2,)
W3 = np.array([[0.8],[-0.5]])                       # (2,1)
b3 = np.array([0.05])                               # (1,)

a1 = sigmoid(W1.T @ x  + b1)      # (3,)  -> [0.6722 0.6004 0.6711]
a2 = sigmoid(W2.T @ a1 + b2)      # (2,)  -> [0.5532 0.6274]
y  = sigmoid(W3.T @ a2 + b3)      # (1,)  -> [0.5446]
print(y)
```

Part 7 — Pro level

7.1 Activation functions beyond the sigmoid

We used the sigmoid everywhere to stay consistent with the lecture. Modern networks mix several activation functions; the *activation* is what makes the network non-linear. **Without a non-linear activation, stacking layers collapses into one linear map** ($W_2(W_1x) = (W_2W_1)x$), no matter how deep.



Common activation functions: Step (old Perceptron), Sigmoid (0–1), Tanh (−1 to 1), ReLU (0 for negatives, identity for positives).

Activation	Formula	Typical use	Note
Sigmoid	$1 / (1 + e^{-z})$	Binary output layer	Can saturate → slow learning
Tanh	$(e^{z} - e^{-z})/(e^{z} + e^{-z})$	Hidden layers (older)	Zero-centred
ReLU	max(0, z)	Default for hidden layers	Fast, avoids saturation on the positive side
Softmax	$e^{z_k} / \Sigma\, e^{z_j}$	Multi-class output	Probabilities sum to 1

7.2 Processing many rows at once (batches)

In practice you never push one row at a time. Stack *m* rows into a matrix **X** of shape (m × 4) and the same code computes all predictions in one go (this is why GPUs are so useful). In the row convention:

$$A^{[l]} = \sigma(A^{[l-1]}W^{[l]} + b^{[l]}): \quad (m \times 4)(4 \times 3) \to (m \times 3) \to (m \times 2) \to (m \times 1)$$

The bias vector (1 × n) is automatically **broadcast** (added to every row).

7.3 Backpropagation preview (what comes next)

After forward propagation we compare ŷ with the truth y using the loss L, then compute how each weight influenced the loss using the **chain rule**, and nudge it downhill (**gradient descent**):

$$w \leftarrow w - \eta \frac{\partial L}{\partial w} \qquad (\eta = \text{learning rate})$$

For a sigmoid output with log-loss, the maths simplifies beautifully:

$$\frac{\partial L}{\partial z^{[L]}} = \hat{y} - y$$

Another useful fact: the sigmoid's derivative is $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, with a maximum of 0.25 — in very deep sigmoid networks gradients shrink layer after layer (the **vanishing-gradient problem**), which is one reason ReLU became popular.

7.4 The same network in Keras (3 lines of real work)

The lecture series next trains networks on real datasets with Keras, which handles backpropagation behind the scenes. Our 4-3-2-1 network:

Python (Keras)

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Input(shape=(4,)),                # 4 input columns
    layers.Dense(3, activation="sigmoid"),   # hidden layer 1 : 4*3+3 = 15 params
    layers.Dense(2, activation="sigmoid"),   # hidden layer 2 : 3*2+2 =  8 params
    layers.Dense(1, activation="sigmoid"),   # output layer   : 2*1+1 =  3 params
])
model.summary()                              # Total params: 26

model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2)
print(model.predict(X_new))                  # forward propagation
```

7.5 Practical pitfalls & professional habits

- **Scale your inputs** (standardise or min-max). Raw values like IQ = 87 can push z into the saturated region of the sigmoid.
- **Random initialisation matters.** If all weights start equal, all nodes in a layer learn the same thing (symmetry). Use small random values (Glorot / He initialisation).
- **Always count parameters** and compare with your amount of data: huge networks on small data *overfit*.
- **Start small, then grow**: add nodes or layers only while validation performance improves.
- **Check shapes** first when debugging — most forward-pass bugs are shape mismatches.
- **Match output layer to task**: 1 sigmoid node (binary), softmax with k nodes (multi-class), linear node (regression).

7.6 Hands-on demo: TensorFlow Playground

The lecture closes with a demo in **TensorFlow Playground** (playground.tensorflow.org) — a free browser tool where you build and train small networks and watch boundaries form. Try these experiments to see every idea of this tutorial live:

#	Experiment	What you will see
1	Pick the **circle** or **XOR** dataset, set **0 hidden layers**, press play	Only a straight line — the Perceptron limit (Part 1)
2	Add **1 hidden layer with 2 neurons**, activation = sigmoid or tanh	A bent boundary; hover over each neuron to see *its own line* (Part 3)
3	Increase to **3, 4, 8 neurons**	Smoother, more complex boundaries (Way 1)
4	Choose the hard **spiral** dataset; add a 2nd and 3rd hidden layer	Only deeper networks capture it (Way 4)
5	Add more input **features** (x_1^2, x_2^2, x_1x_2, sin…)	Changing the input layer (Way 2)
6	Watch the thickness / colour of connections	Blue vs orange weights = positive vs negative

Part 8 — Summary, cheat-sheet, exercises & glossary

8.1 The whole story in ten lines

- A Perceptron draws only a straight line ⇒ fails on non-linear data.
- Use sigmoid Perceptrons (logistic-regression style) so each outputs a probability.
- Take two Perceptrons, **add** their outputs (superimposition) and apply **sigmoid** (smoothing).
- Make the addition weighted and add a bias ⇒ the combiner is itself a Perceptron.
- Three Perceptrons in layers = Multi-Layer Perceptron (input, hidden, output layers).
- Architecture can be changed 4 ways: more hidden nodes, more input nodes, more output nodes, more hidden layers.
- Enough hidden units/layers + training ⇒ **universal function approximator**.
- Parameters per layer = $n_{in} \times n_{out} + n_{out}$.
- Forward propagation: $a^{[l]} = \sigma((W^{[l]})^T a^{[l-1]} + b^{[l]})$, layer after layer.
- Backpropagation (next) adjusts all weights and biases to reduce the loss.

8.2 Cheat-sheet

Concept	Formula / rule
Weighted sum	$z = w_1x_1 + \ldots + w_nx_n + b$
Sigmoid	$\sigma(z) = 1/(1+e^{-z})$, $\sigma(0)=0.5$
Two probabilities	$P(\text{No}) = 1 - P(\text{Yes})$
Log-loss	$-[y \log \hat{y} + (1-y) \log(1-\hat{y})]$

Concept	Formula / rule
Layer parameters	$n_{in} \cdot n_{out} + n_{out}$
Weight matrix shape	$W^{[l]}$: $(n_{l-1} \times n_l)$; transposed: $(n_l \times n_{l-1})$
Forward step	$a^{[l]} = \sigma((W^{[l]})^T a^{[l-1]} + b^{[l]})$
Matrix multiply rule	$(a \times b) \cdot (b \times c) \to (a \times c)$
Input nodes	= number of input columns
Output nodes	1 (binary) or k (k classes)
Gradient descent	$w \leftarrow w - \eta \, \partial L / \partial w$

8.3 Practice exercises

#	Question
1	A sigmoid Perceptron has w = (5, 10), b = 3. Find z for CGPA 8.7, IQ 87 (done in the text). What does σ(z) tell you?
2	Hidden outputs are 0.6 and 0.3, output weights 4 and −2, bias 1. Compute z and $\hat{y}$.
3	How many trainable parameters has a 3 → 5 → 4 → 1 network?
4	How many parameters has a 784 → 128 → 64 → 10 network?
5	For a network with 10 inputs and 6 hidden nodes, give the shape of W^1 and of $(W^1)^T$.
6	Network 2 → 2 → 1, input x = (1, 0), W^1 = [[1, −1],[2, 1]] (rows = inputs), b^1 = (0, 0), W^2 = [[1],[1]], b^2 = −1. Compute $\hat{y}$ by hand.
7	Why can't you remove the sigmoid from the hidden layer and still capture non-linearity?

Answers

#	Answer
1	z = 916.5 ⇒ σ(z) ≈ 1.0: the model is essentially certain the student is placed (inputs unscaled, so saturated).
2	$z = 0.6 \cdot 4 + 0.3 \cdot (-2) + 1 = 2.4 - 0.6 + 1 = 2.8 \Rightarrow \hat{y} = \sigma(2.8) \approx 0.943$.
3	$(3 \cdot 5 + 5) + (5 \cdot 4 + 4) + (4 \cdot 1 + 1) = 20 + 24 + 5 =$ **49**.
4	$(784 \cdot 128 + 128) + (128 \cdot 64 + 64) + (64 \cdot 10 + 10) = 100{,}480 + 8{,}256 + 650 =$ **109,386**.
5	W^1 is (10 × 6); $(W^1)^T$ is (6 × 10).
6	$z^1 = (W^1)^T x = [1, -1] \Rightarrow a^1 = [\sigma(1), \sigma(-1)] = [0.731, 0.269]$; $z^2 = 0.731 + 0.269 - 1 = 0 \Rightarrow \hat{y} = \sigma(0) =$ **0.5**.
7	Without a non-linearity, every layer is a linear map, and a composition of linear maps is still linear ($W_2(W_1 x) = (W_2 W_1)x$) — only a straight boundary.

8.4 Glossary

Term	Meaning
Perceptron / neuron / node	A unit that computes σ(w·x + b).
Decision boundary	The set of points where the model's probability is exactly 0.5.
Linear / non-linear	Separable by a straight line / needs a curve.
Sigmoid	Squashing function giving a value between 0 and 1.
Log-loss	Loss for probabilistic binary classification.
Hidden layer	A layer between input and output.
Architecture	How nodes and layers are connected.
Weight / bias	Learnable numbers on connections / in nodes.
Activation (a)	The output of a layer after the sigmoid.
Forward propagation	Computing ŷ from an input, layer by layer.
Backpropagation	Sending the error backward to update the weights.
Trainable parameters	All weights and biases that are learned.
Universal function approximator	A network that can approximate any function given enough capacity.
Epoch / batch	One pass over all data / a group of rows processed together.
Keras	High-level Python library that builds and trains neural networks.

→ What's next
Next in the series: training networks on real datasets with Keras, then the full mathematics of **backpropagation** — now you already know exactly what the forward pass does.